CVPR 2026 Best Paper

# Efficiently Reconstructing Dynamic Scenes One D4RT at a Time

IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) 2026
Colorado Convention Center, Denver, Colorado, United States June 2026

Chuhan Zhang, Guillaume Le Moing, Skanda Koppula, Ignacio Rocco, Liliane Momeni, Junyu Xie, Shuyang Sun, Rahul Sukthankar, Joëlle K. Barral, Raia Hadsell, Zoubin Ghahramani, Andrew Zisserman, Junlin Zhang, Mehdi S. M. Sajjadi

主要單位： Google DeepMind 。 Skanda Koppula 另掛 University College London； Andrew Zisserman 另掛 University of Oxford 。
Junyu Xie 標示為 University of Oxford ，並註明於 Google DeepMind 實習期間完成。通訊作者： Mehdi S. M. Sajjadi

---

報告人 第 17 組

**廖晢勛 B1229009 林威宇 B1229010**

指導教授 陳仁暉 長庚大學 電腦視覺

專案頁 https://d4rt-paper.github.io/

# 動態 4D 重建

論文 Figure 1

## 點雲

每個可見點離相機多遠，放到同一個三維空間

## 三維軌跡

同一個點隨著時間跑到哪

## 相機參數

每一幀的位置、朝向，還有鏡頭內參

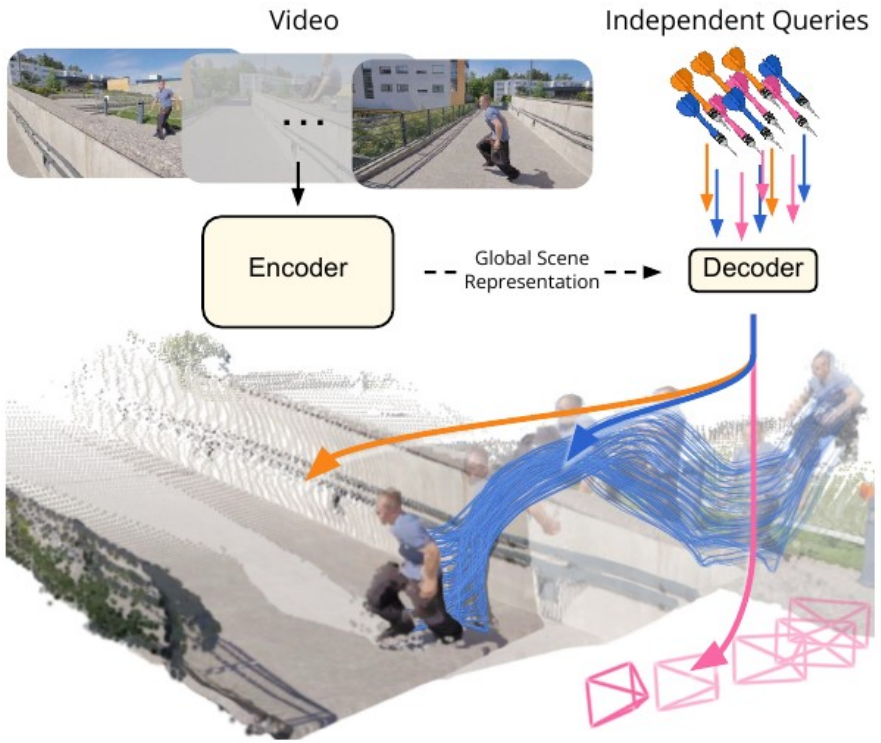



Figure 1. **D4RT** is a unified, efficient, feedforward method for ***D**ynamic **4D** **R**econstruction and **T**racking*, unlocking a variety of outputs including point cloud (), point tracks (), camera parameters () through a single interface.

橘：點雲　藍：軌跡　粉：相機

# 跟以前的方法比

論文 Table 2；沒贏的項目見 Table 4、 5、 6

| Model | Task | | Functionality | | Architecture | |
|---|---|---|---|---|---|---|
| | 3D Reconstruction | Dynamic Correspondence | Flexible Ref. Frames | Sparse Decoding | Global Context | Single Decoder |
| MegaSaM [32] | ✓ | ✗ | ✗ | ✗ | ✗ | ✗ |
| DUSt3R[54] | ✓ | ✗ | ✗ | ✗ | ✗ | ✗ |
| VGGT [51] | ✓ | ✗ | ✗ | ✗ | ✓ | ✗ |
| $\pi^3$ [57] | ✓ | ✗ | ✓ | ✗ | ✓ | ✗ |
| St4RTrack [13] | ✓ | ✓ | ✗ | ✗ | ✗ | ✗ |
| STv2 [59] | ✓ | ✓ | ✗ | ✓ | ✓ | ✗ |
| **D4RT (Ours)** | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |

## 這篇不一樣的地方

單一前饋模型
深度、姿勢、追蹤同一套
以前：多個模型拼接，或測試時再優化

只解有問到的點
所以比較快（ Figure 3 、 Table 3 ）

會動的場景
$\pi^3$ 這類方法會失敗（ Figure 4 ）

## 沒贏的地方

ADT 、沒給真實內參： SpatialTrackerV2 較好
Bonn 深度、 ScanNet 旋轉誤差： $\pi^3$ 較好

# 幾個會用到的名詞

### 深度、點雲

像素離相機多遠；放回三維就是點雲

### 內參

鏡頭規格：焦距、主點

### 外參

相機的位置與朝向

### 對應、追蹤

不同幀的像素是不是同一個點；串起來就是軌跡

### 相機座標 / 世界座標

用某一幀的眼睛寫，或整段影片共用一個座標系

### 尺度對齊

單鏡頭分不出近的小東西和遠的大東西；評估前先對齊尺度

# 方法：用 query 問三維位置

論文 Figure 2

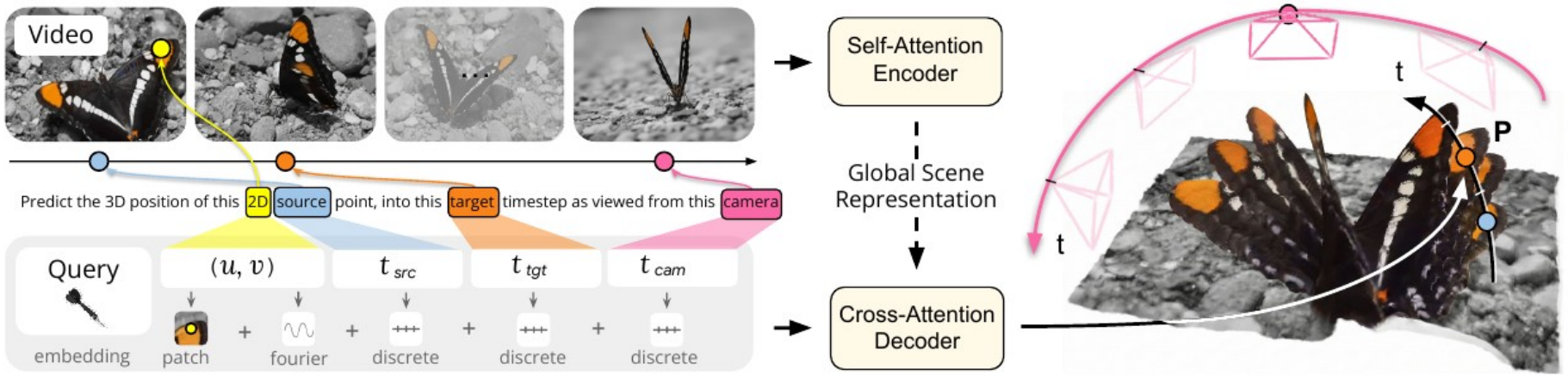



Figure 2. **D4RT model overview** – A global self-attention encoder first transforms the input video into the latent *Global Scene Representation* $F$, which is passed to a lightweight decoder. The decoder can be independently queried for the *3D position* $\mathbf{P}$ of any given 2D point $(u, v)$ from the source timestep $t_{\text{src}}$ at target timestep $t_{\text{tgt}}$ in camera coordinate $t_{\text{cam}}$, unlocking full decoding at any point in space and time. The query also contains an embedding of the local video patch centered around $(u, v)$, providing additional spatial context.

**q = (u, v, t_src, t_tgt, t_cam)　→　P = D(q, F)**

來源幀上的二維點，在目標時間、用指定的相機座標系來寫，三維位置在哪
三個時間可以不一樣　每個 query 分開回答

# 五種輸出，同一套 query

論文 Table 1

| Task | Query | | | | |
|---|---|---|---|---|---|
| | $u$ | $v$ | $t_{src}$ | $t_{tgt}$ | $t_{cam}$ |
| Point Track | Fixed | Fixed | Fixed | $1 \ldots T$ | |
| Point Cloud | $1 \ldots W$ | $1 \ldots H$ | $1 \ldots T$ | | Fixed |
| Depth Map | $1 \ldots W$ | $1 \ldots H$ | $1 \ldots T$ | | |
| Extrinsics | $1 \ldots h$ | $1 \ldots w$ | Fixed | | $1 \ldots T$ |
| Intrinsics | $1 \ldots h$ | $1 \ldots w$ | $1 \ldots T$ | | |

## 怎麼問

軌跡：固定 (u, v)， t_tgt 與 t_cam 走 1⋯T

點雲： t_src = t_tgt ，一起走 1⋯T ， t_cam 固定

深度：三個時間同一幀，取 Z

外參：同一批點問兩個座標系， Umeyama （一次 3×3 SVD ）

內參：針孔、主點 (0.5, 0.5)
fx = pz (u − 0.5) / px
再取中位數

沒有另外的姿勢輸出頭

# 編碼器與解碼器

## 編碼器 E

ViT-g， 40 層，約 10 億參數
時空 patch 2×16×16
交錯：幀內局部注意力、跨幀全域注意力
先縮成正方形；原長寬比另做一個 token
初始化：公開的 VideoMAEv2

## 解碼器 D

8 層 cross-attention， 1.44 億參數
Query： Fourier(u, v) + 三個時間 + 9×9 RGB
query 之間不做 self-attention
早期實驗：打開後推論表現下降
一致性寫進全域特徵 F

訓練： 48 幀、 256×256 、每次 2048 個 query 、 AdamW 、 50 萬步、 64 顆 TPU 、兩天多

# 為什麼比較快

論文 Figure 3 、 Table 3 、 Algorithm 1

## Algorithm 1

先算一次全域特徵 F
佔用格記住哪些時空位置走過了
只從沒走過的像素長新軌跡（約 5–15 倍）

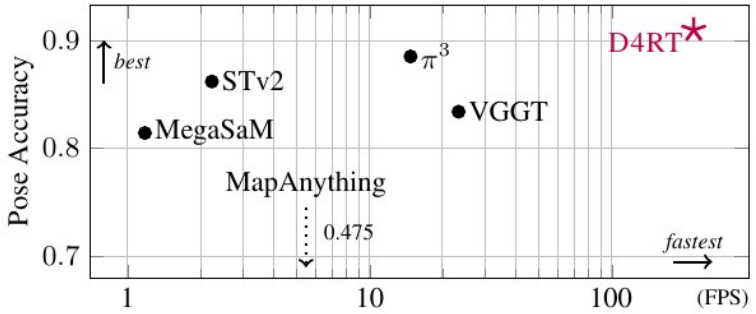



Figure 3. **Pose accuracy *vs.* speed** – D4RT achieves over 200 FPS for pose estimation on an A100 GPU, delivering superior accuracy while running 9× faster than VGGT and 100× faster than MegaSaM. Accuracy is defined as *1 – error*, averaged over ATE / RTE / RPE on Sintel and ScanNet.

Figure 3： A100 姿勢估計 >200 FPS
比 VGGT 快 9 倍，比 MegaSaM 快 100 倍

| Method | Max. Track Count @ Target FPS | | | |
|---|---|---|---|---|
| | 60 FPS | 24 FPS | 10 FPS | 1 FPS |
| DELTA [35] | 0 | 5 | 408 | 5,770 |
| SpatialTrackerV2 [59] | 29 | 84 | 219 | 2,290 |
| **D4RT (Ours)** | 550 | 1,570 | 3,890 | 40,180 |

Table 3　維持 24 FPS 、單張 A100
D4RT 1,570 條
SpatialTrackerV2 84 條

論文宣稱 18–300 倍
對 STv2 約 18 倍；對 DELTA 約 7–314 倍

# 重建結果

論文 Figure 4

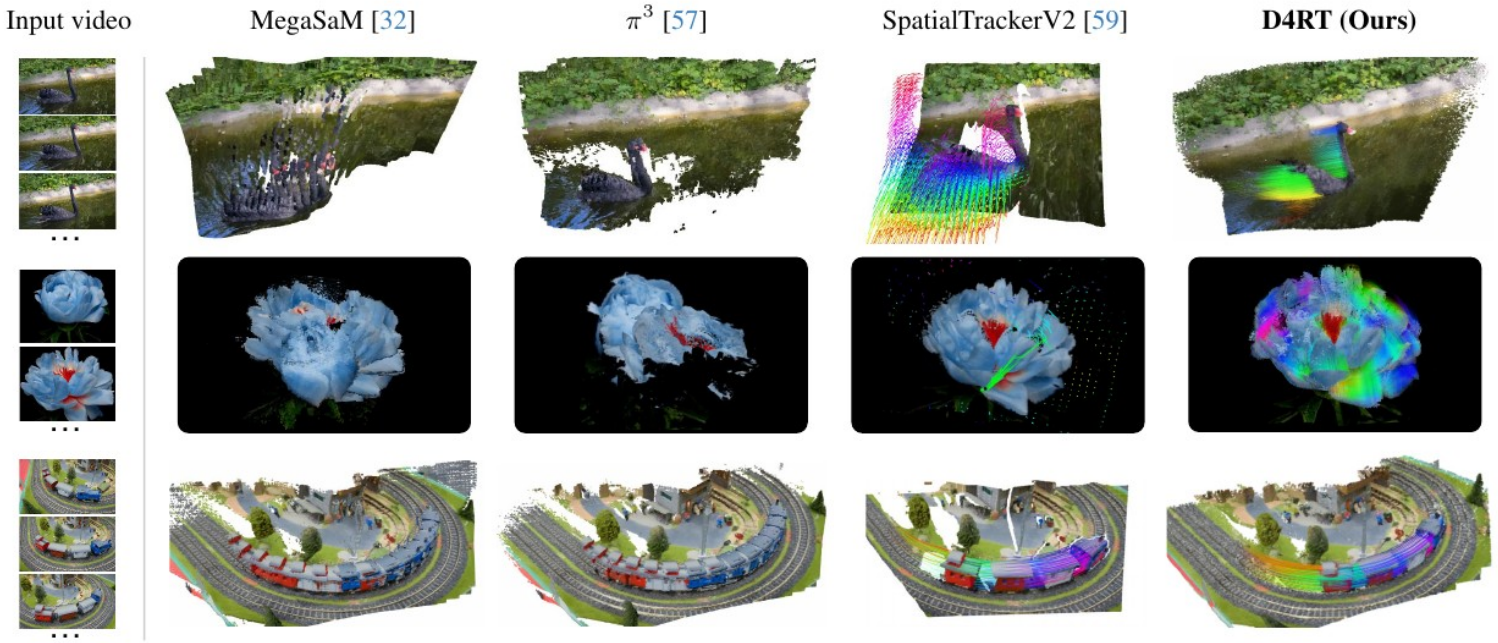



Figure 4. **Reconstruction results across methods** – Pure reconstruction methods (MegaSaM and $\pi^3$) are only able to accumulate point clouds of all pixels; exhibiting clear failure cases in dynamic scenes. For example, the swan is repeated in MegaSaM's reconstruction, and $\pi^3$ is failing entirely to reconstruct the flower. SpatialTrackerV2, a state-of-the-art tracking method, successfully captures dynamics, however its design only allows tracking points from *one* frame, leaving gaps in the reconstruction (behind the swan and train). D4RT is the only method that successfully reconstructs a full 4D representation of the scene including *all* pixels of the video.

MegaSaM： 天鵝重複印上　　π³： 論文寫完全重建不出花　　SpatialTrackerV2： 單幀追蹤會缺一塊　　D4RT： 補上各幀的像素

# 追蹤（Table 4）

TAPVid-3D；愈高愈好的是 AJ、APD3D、OA

| w/ GT intrin. | Method | TAPVid-3D | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | Camera Coordinate 3D tracking | | | | | | | | | World Coordinate 3D tracking | | | |
| | | DriveTrack | | | ADT | | | PStudio | | | DriveTrack | | ADT | |
| | | AJ ↑ | $APD_{3D}$ ↑ | OA ↑ | AJ ↑ | $APD_{3D}$ ↑ | OA ↑ | AJ ↑ | $APD_{3D}$ ↑ | OA ↑ | $APD_{3D}$ ↑ | L1 ↓ | $APD_{3D}$ ↑ | L1 ↓ |
| ✗ | St4RTrack [13] | - | - | - | - | - | - | - | - | - | 0.020 | 0.499 | 0.062 | 0.839 |
| | CoTracker3 [26] + UniDepthV2 [39] | 0.038 | 0.062 | 0.856 | 0.102 | 0.146 | 0.937 | 0.119 | 0.176 | 0.862 | - | - | - | - |
| | CoTracker3 [26] + VGGT [51] | 0.129 | 0.189 | 0.856 | 0.132 | 0.190 | 0.937 | 0.045 | 0.070 | 0.862 | 0.205 | 0.170 | 0.192 | 0.736 |
| | SpatialTrackerV2 [59] | 0.064 | 0.100 | 0.865 | 0.260 | 0.342 | 0.936 | 0.097 | 0.152 | 0.805 | 0.101 | 0.139 | 0.336 | 0.238 |
| | **D4RT (Ours)** | 0.257 | 0.345 | 0.875 | 0.240 | 0.319 | 0.926 | 0.138 | 0.186 | 0.897 | 0.373 | 0.020 | 0.319 | 0.096 |
| ✓ | CoTracker3 [26] + UniDepthV2 [39] | 0.147 | 0.225 | 0.856 | 0.173 | 0.254 | 0.937 | 0.205 | 0.311 | 0.862 | - | - | - | - |
| | CoTracker3 [26] + VGGT [51] | 0.245 | 0.342 | 0.856 | 0.175 | 0.250 | 0.937 | 0.215 | 0.318 | 0.862 | 0.292 | 0.160 | 0.234 | 0.755 |
| | DELTA [35] | 0.170 | 0.238 | 0.874 | 0.199 | 0.258 | 0.879 | 0.175 | 0.261 | 0.760 | - | - | - | - |
| | SpatialTrackerV2 [59] | 0.195 | 0.275 | 0.865 | 0.303 | 0.404 | 0.936 | 0.175 | 0.270 | 0.805 | 0.201 | 0.117 | 0.378 | 0.263 |
| | **D4RT (Ours)** | 0.304 | 0.410 | 0.875 | 0.307 | 0.408 | 0.926 | 0.372 | 0.498 | 0.897 | 0.470 | 0.017 | 0.398 | 0.093 |

Table 4. **4D reconstruction and tracking** – We evaluate 3D tracking capability on dynamic videos, with tracks predicted in both local camera coordinates (left) and world coordinates (right). Our model achieves superior performance compared to the prior state-of-the-art.

無真實內參、DriveTrack AJ　0.257（次佳 0.129）
世界座標 L1、DriveTrack　0.020（對照方法 > 0.1）
ADT、無真實內參：SpatialTrackerV2 的 AJ 0.260、APD3D 0.342，高於 D4RT 的 0.240、0.319

# 點雲、深度、姿勢

論文 Table 5 、 Table 6

| Method | Point Cloud | | Video Depth | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | Sintel | ScanNet | Sintel | | ScanNet | | KITTI | | Bonn | |
| | L1 ↓ | L1 ↓ | AbsRel (S) ↓ | AbsRel (SS) ↓ | AbsRel (S) ↓ | AbsRel (SS) ↓ | AbsRel (S) ↓ | AbsRel (SS) ↓ | AbsRel (S) ↓ | AbsRel (SS) ↓ |
| MegaSaM [32] | 1.531 | 0.072 | 0.342 | 0.249 | 0.050 | 0.047 | 0.109 | 0.101 | 0.056 | 0.056 |
| VGGT [51] | 1.582 | 0.063 | 0.318 | 0.247 | 0.044 | 0.040 | 0.094 | 0.067 | 0.055 | 0.051 |
| MapAnything [27] | 1.718 | 0.064 | 0.397 | 0.306 | 0.043 | 0.035 | 0.096 | 0.090 | 0.076 | 0.049 |
| SpatialTrackerV2 [59] | 1.375 | 0.036 | 0.209 | 0.175 | 0.027 | 0.025 | 0.075 | 0.064 | 0.042 | 0.978 |
| $\pi^3$ [57] | 1.139 | 0.030 | 0.241 | 0.163 | 0.021 | 0.019 | 0.055 | 0.053 | 0.033 | 0.032 |
| **D4RT (Ours)** | 0.768 | 0.028 | 0.171 | 0.148 | 0.020 | 0.018 | 0.055 | 0.051 | 0.036 | 0.036 |

Table 5. **Video depth and point map estimation** – Quantitative results for both video depth and point map estimation across four benchmarks. D4RT achieves top-tier performance on the depth estimation task under both scale-only (S) and scale-and-shift (SS) alignments.

點雲 L1　Sintel 0.768 （$\pi^3$ 為 1.139）； ScanNet 0.028
深度 AbsRel　Sintel 0.171 / 0.148
姿勢 ATE　Sintel 0.065 、 ScanNet 0.014 （都是表上最好）
Re10K Pose AUC　83.5 （次佳 78.7）

沒贏： Bonn 深度是 $\pi^3$ 的 0.033 / 0.032 ，對 D4RT 的 0.036 / 0.036 ； ScanNet RPE-R 是 $\pi^3$ 的 0.291 ，對 0.302

# 消融實驗

論文 Figure 6 、 Table 7–9

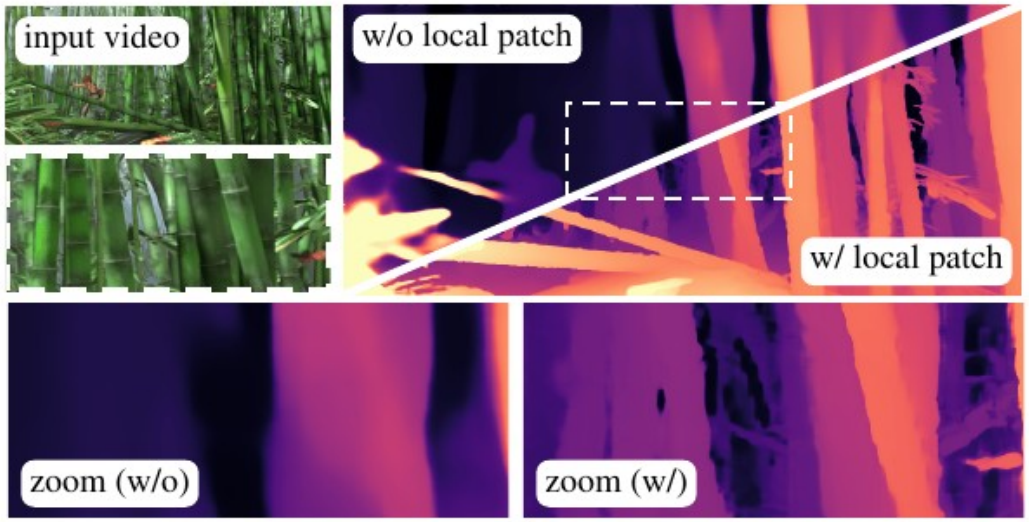



Figure 6. **Preservation of low-level details** – We ablate the effect of including local patch information into the model's queries, visualizing the resulting depth maps on an example from the Sintel dataset. We find that the local RGB patches help preserve fine-grained details and produce sharper object boundaries.

Table 7　9×9 patch （ViT-L, Sintel）
深度 AbsRel　0.366 → 0.302
相機 ATE　0.173 → 0.091

Table 8　編碼器放大
深度、 RPE-R 大致變好
ViT-g 的 ATE 0.078，沒贏過 ViT-H 的 0.070

Table 9　輔助損失
二維位置、法向：幫深度
拿掉信心項： ATE +0.126

# 限制

論文沒有獨立的 Limitations 小節；以下是我們整理的

不是每一格都第一
ADT （尤其沒給真實內參時）、 Bonn 深度、 ScanNet 的 RPE-R

訓練混有內部資料
作者說只用公開資料仍是最先進，但那些數字在附錄，正式 11 頁看不到

訓練設定是 48 幀、 256×256
更長或更高解析的影片，正文沒有系統分析

# 小結

同一個 query：深度、點雲、追蹤、相機參數

只解有問到的點，所以比較快
A100 上姿勢估計，論文寫超過 200 FPS

多數基準領先，但不是每一格都贏

專案頁　https://d4rt-paper.github.io/
謝謝老師，歡迎提問。